

ΨΗΦΙΑΚΗ ΕΠΕΞΕΡΓΑΣΙΑ ΣΗΜΑΤΩΝ

ΠΡΟΑΙΡΕΤΙΚΗ ΕΡΓΑΣΙΑ



ΔΗΜΟΚΡΙΤΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΡΑΚΗΣ

ΤΜΗΜΑ
ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
& ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Ιατρού Κωνσταντίνος 58071

ΠΙΝΑΚΑΣ

ΠΕΡΙΕΧΟΜΕΝΩΝ

Εισαγωγή	2
Ερώτημα Α.....	3
Ερώτημα Β.....	7
Ερώτημα C.....	11
Ερώτημα D.....	13
Ερώτημα E.....	17
Ερώτημα F	21

Εισαγωγή

Για να μην παράγω συνεχώς ένα τυχαίο διάνυσμα με λευκό γκαουσιανό θόρυβο, το οποίο θα έχει ως αποτέλεσμα να μην είναι όμοιο σε όλα τα ερωτήματα, γράφηκε script αρχείο με τίτλο **Noisy_wave_creation.m** μέσα στο οποίο δημιουργώ το διάνυσμά με το λευκό θόρυβο και το προσθέτω στο αρχείο ήχου που μας δόθηκε (guit1.wav). Στη συνέχεια εξάγω τόσο τον λευκό θόρυβο όσο και το ενθόρυβο διάνυσμα σε μορφή wav για να χρησιμοποιώ σταθερά το ίδιο αρχείο με θόρυβο σε όλα τα ερωτήματα.

```

1 - clear; clc;
2 - [guit1, Fs] = audioread('guit1.wav');
3 - n = length(guit1);
4 - %plot(1:length(y), y)
5
6 - %Create Gaussian White Noise with variance 0.1
7 - white_noise = 0.01*randn(n, 1);
8
9 - %Check variance rounded to first 2 decimal digits
10 - fprintf("Typical Deviation value = ");
11 - disp(std(white_noise));
12
13 - %Add noise to audio file
14 - noisy_y = guit1 + white_noise;
15
16 - audiowrite('white_noise.wav', white_noise, Fs);
17 - audiowrite('noisy.wav', noisy_y, Fs);

```

Το διάνυσμα με τον λευκό γκαουσιανό θόρυβο παράγεται στη γραμμή 7. Ο πολλαπλασιασμός με το 0.01 γίνεται γιατί στην εκφώνηση ζητείται ο λευκός θόρυβος να έχει τυπική απόκλιση $\sigma = 0.01$.

Το ενθόρυβο σήμα του αρχείο που μας δόθηκε δημιουργείται στην γραμμή 14 όπου προστίθεται ο θόρυβος με το διάνυσμα του guit1.wav.

Στην συνέχεια τα εξάγω σε μορφή wav για να τα εισάγω σε κάθε ξεχωριστό script που θα γραφεί για κάθε ερώτημα.

Ερώτημα Α

Για το ερώτημα α γράφηκε το παρακάτω script αρχείο που υλοποιεί και τα 3 φίλτρα Wiener με τάξεις $p = 10, 20$ και 30 :

```

1 - clear; clc; close all
2 - %Task a
3 - %Load wav files
4 - [y, Fs] = audioread('guit1.wav');
5 - white_noise = audioread('white_noise.wav');
6 - noisy_y = audioread('noisy.wav');
7 - n = length(y);
8 - snr_vec = 20 * log10(norm(y)/norm(noisy_y-y));
9 - %Plot a combined graph of noisy and starting waves between values 1000 and
10 - %1250
11 -
12 - tiledlayout(2,1);
13 - ax1 = nexttile;
14 - plot(ax1, 750:1500, y(750:1500))
15 - axis([1000, 1250, -0.6, 0.6])
16 - title(ax1, 'Original waveform')
17 -
18 - ax2 = nexttile;
19 - plot(ax2, 750:1500, noisy_y(750:1500))
20 - axis([1000, 1250, -0.6, 0.6])
21 - title(ax2, 'Noisy waveform')
22 -
23 - %Wiener order p=10
24 - p = 10;
25 - rxx = xcorr(noisy_y);
26 - rxd = xcorr(noisy_y, y);
27 - rxx = rxx(n:n+p-1);
28 - rxd = rxd(n:n+p-1);
29 -
30 - Rxx = toeplitz(rxx);
31 - w = Rxx\rxd; % It is the same as inv(Rxx)*rxd
32 -
33 - z10 = filter(w, 1, noisy_y);
34 - snr_vec = [snr_vec 20 * log10(norm(y)/norm(z10-y))];
35 - audiowrite('wiener10_a.wav', z10, Fs);
36 -
37 - %Wiener order p=20
38 - p = 20;
39 - rxx = xcorr(noisy_y);
40 - rxd = xcorr(noisy_y, y);
41 - rxx = rxx(n:n+p-1);
42 - rxd = rxd(n:n+p-1);
43 -
44 - Rxx = toeplitz(rxx);
45 - w = Rxx\rxd;
46 -
47 - z20 = filter(w, 1, noisy_y);
48 - snr_vec = [snr_vec 20 * log10(norm(y)/norm(z20-y))];
49 - audiowrite('wiener20_a.wav', z20, Fs);
50 -
51 - %Wiener order p=30
52 - p = 30;
53 - rxx = xcorr(noisy_y);
54 - rxd = xcorr(noisy_y, y);
55 - rxx = rxx(n:n+p-1);
56 - rxd = rxd(n:n+p-1);
57 -
58 - Rxx = toeplitz(rxx);
59 - w = Rxx\rxd;
60 -
61 - z30 = filter(w, 1, noisy_y);
62 - snr_vec = [snr_vec 20 * log10(norm(y)/norm(z30-y))];
63 -
64 - audiowrite('wiener30_a.wav', z30, Fs);

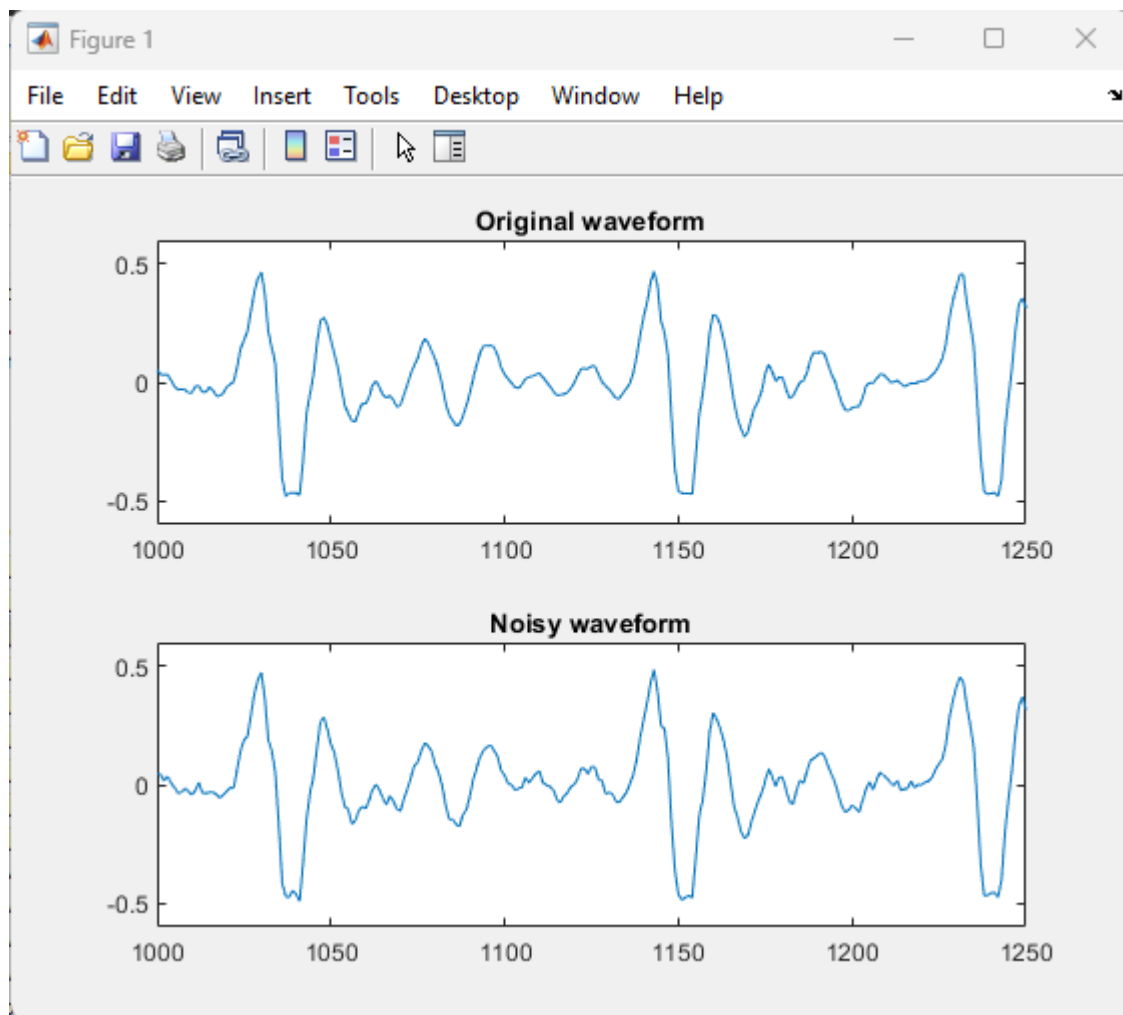
```

Αυτό που γίνεται στο αρχείο αυτό είναι:

Αρχικά εισάγονται το αρχικό αρχείο guit1.wav και το ενθόρυβο noisy.wav, ως **y** και **noisy_y** αντίστοιχα.

Κρατάω το μήκος των σημάτων στη μεταβλητή **n** και υπολογίζω το SNR του ενθόρυβου σήματος, τοποθετώντας στη μεταβλητή **snr_vec** η οποία θα είναι ένας πίνακας με όλα τα SNR.

Στην συνέχεια γίνονται plot σε ένα παράθυρο ένα μέρος των δύο σημάτων για να συγκριθούν.



Μετά από αυτό έχω την υλοποίηση των φίλτρων Wiener και την έξοδο του καθενός όταν τους εφαρμόζεται το ενθόρυβο σήμα. Η έξοδος του κάθε φίλτρου εκχωρείται στην αντίστοιχη μεταβλητή (**z10**, **z20**, **z30**).

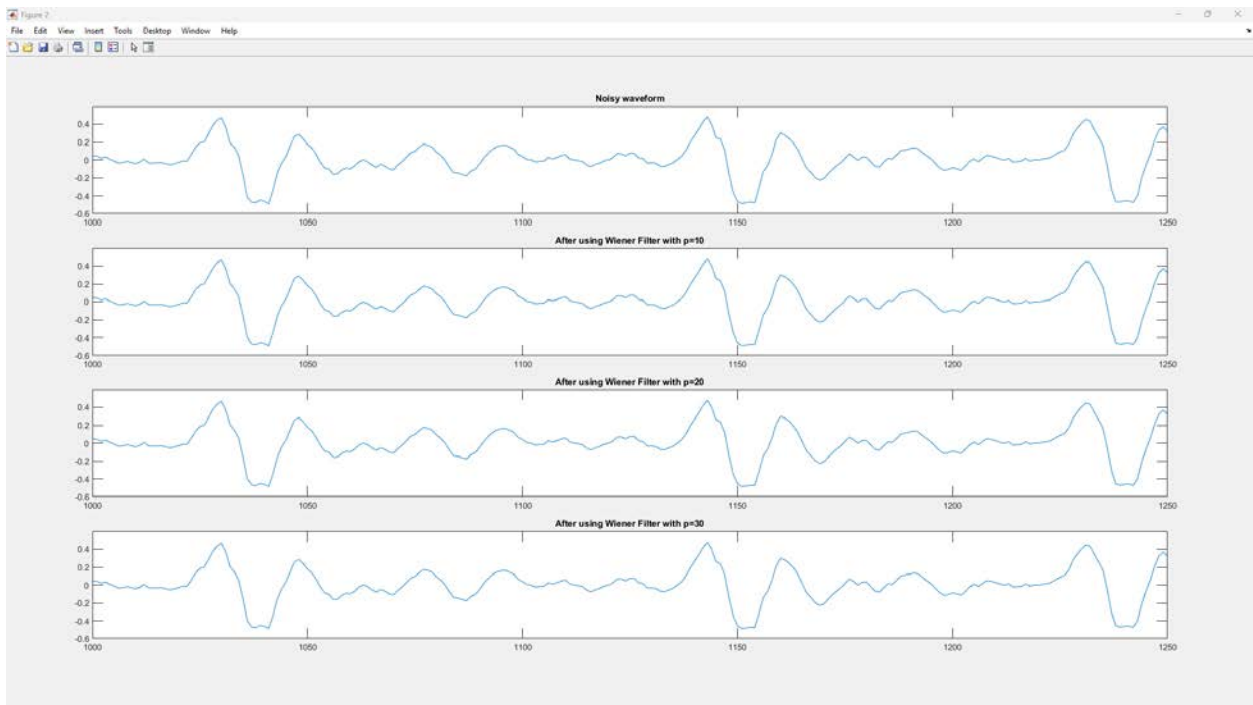
Τέλος εξάγουμε τις μεταβλητές ως αρχεία wav με τα αντίστοιχα ονόματα (**wiener10_a.wav**, **wiener20_a.wav**, **wiener30_a.wav**) και τυπώνω ένα plot με κοινό διάστημα του ενθόρυβου σήματος και των εξόδων των φίλτρων μέσω του παρακάτω μπλοκ κώδικα:

```

65 - figure
66 - tiledlayout(4,1);
67 - ax1 = nexttile;
68 - plot(ax1, 750:1500, noisy_y(750:1500))
69 - axis([1000, 1250, -0.6, 0.6])
70 - title(ax1, 'Noisy waveform')
71
72 - ax2 = nexttile;
73 - plot(ax2, 750:1500, z10(750:1500))
74 - axis([1000, 1250, -0.6, 0.6])
75 - title(ax2, 'After using Wiener Filter with p=10')
76
77 - ax3 = nexttile;
78 - plot(ax3, 750:1500, z20(750:1500))
79 - axis([1000, 1250, -0.6, 0.6])
80 - title(ax3, 'After using Wiener Filter with p=20')
81
82 - ax4 = nexttile;
83 - plot(ax4, 750:1500, z30(750:1500))
84 - axis([1000, 1250, -0.6, 0.6])
85 - title(ax4, 'After using Wiener Filter with p=30')

```

Το κοινό διάγραμμα είναι:



Το SNR του ενθόρυβου και τον φιλτραρισμένων σημάτων από τα φίλτρα τάξης 10, 20 και 30 κατά σειρά είναι τα παρακάτω:

```
>> snr_vec  
  
snr_vec =  
  
    19.356519059539792    19.744910369369435    19.754003716699408    19.759117632184594
```

Είναι φανερό πως δεν βλέπουμε κάποια ιδιαίτερα μεγάλη βελτίωση στην έξοδο του σήματος, κάτι που φαίνεται και από την κυματομορφή που δεν έχει φανερές αλλαγές.

Ακόμη κατόπιν ακρόασης των αρχείων ήχου, αυτό που παρατηρείται είναι ότι δεν γίνεται αντιληπτή καμία βελτίωση από το ανθρώπινο αφτί.

Ερώτημα Β

Για το ερώτημα b γράφηκε το παρακάτω script αρχείο που υλοποιεί και τα 3 φίλτρα Wiener με τάξεις $p = 10, 20$ και 30 σε παράθυρα του σήματος guit1.wav:

```

1 - clear; clc; close all;
2 - %Task b
3 - %=====
4 - %Load wav file
5 - [y, Fs] = audioread('guit1.wav');
6 - white_noise = audioread('white_noise.wav');
7 - noisy_y = audioread('noisy.wav');
8 - n = length(y);
9 - %plot(1:length(y), y)
10
11 - snr_vec = 20*log10(norm(y)/norm(noisy_y-y));
12 - win_size = 256;
13 - frames_original = frame_wind(y, win_size, 0.5);
14 - frames_noisy = frame_wind(noisy_y, win_size, 0.5);
15
16 - % Wiener of order p = 10
17 - p = 10;
18
19 - for i=1:size(frames_original, 2)
20 -
21 -     rxx = xcorr(frames_noisy(:,i));
22 -     rxd = xcorr(frames_noisy(:,i), frames_original(:,i));
23 -     rxx = rxx(win_size:win_size+p-1);
24 -     rxd = rxd(win_size:win_size+p-1);
25
26 -     Rxx = toeplitz(rxx);
27 -     w = pinv(Rxx, 0.0001)*rxd; % It is the same as inv(Rxx)*rxd
28 -     z(:, i) = filter(w, 1, frames_noisy(:,i));
29 - end
30
31 - denoised_10 = [frame_recon(z, 0.5)]';
32 - snr_vec = [snr_vec 20*log10(norm(y)/norm(denoised_10(1:n)-y))];
33 - audiowrite('wiener10_b.wav', denoised_10, Fs);
34
35 - %=====
36 - % Wiener of order p = 20
37 - p = 20;
38
39 - for i=1:size(frames_original, 2)
40 -
41 -     rxx = xcorr(frames_noisy(:,i));
42 -     rxd = xcorr(frames_noisy(:,i), frames_original(:,i));
43 -     rxx = rxx(win_size:win_size+p-1);
44 -     rxd = rxd(win_size:win_size+p-1);
45
46 -     Rxx = toeplitz(rxx);
47 -     w = pinv(Rxx, 0.0001)*rxd; % It is the same as inv(Rxx)*rxd
48 -     z(:, i) = filter(w, 1, frames_noisy(:,i));
49 - end
50
51 - denoised_20 = [frame_recon(z, 0.5)]';
52 - snr_vec = [snr_vec 20*log10(norm(y)/norm(denoised_20(1:n)-y))];
53 - audiowrite('wiener20_b.wav', denoised_20, Fs);
54 - audiowrite('noisy.wav', noisy_y, Fs);
55

```

```

56 %-----
57 % Wiener of order p = 30
58 - p = 30;
59
60 - for i=1:size(frames_original, 2)
61
62 -     rxx = xcorr(frames_noisy(:,i));
63 -     rxd = xcorr(frames_noisy(:,i), frames_original(:,i));
64 -     rxx = rxx(win_size:win_size+p-1);
65 -     rxd = rxd(win_size:win_size+p-1);
66
67 -     Rxx = toeplitz(rxx);
68 -     w = pinv(Rxx, 0.0001)*rxd; % It is the same as inv(Rxx)*rxd
69 -     z(:, i) = filter(w, 1, frames_noisy(:,i));
70 - end
71
72 - denoised_30 = [frame_recon(z, 0.5)]';
73 - snr_vec = [snr_vec 20*log10(norm(y)/norm(denoised_30(1:n)-y))];
74 - audiowrite('wiener30_b.wav', denoised_30, Fs);
75 - audiowrite('noisy.wav', noisy_y, Fs);

```

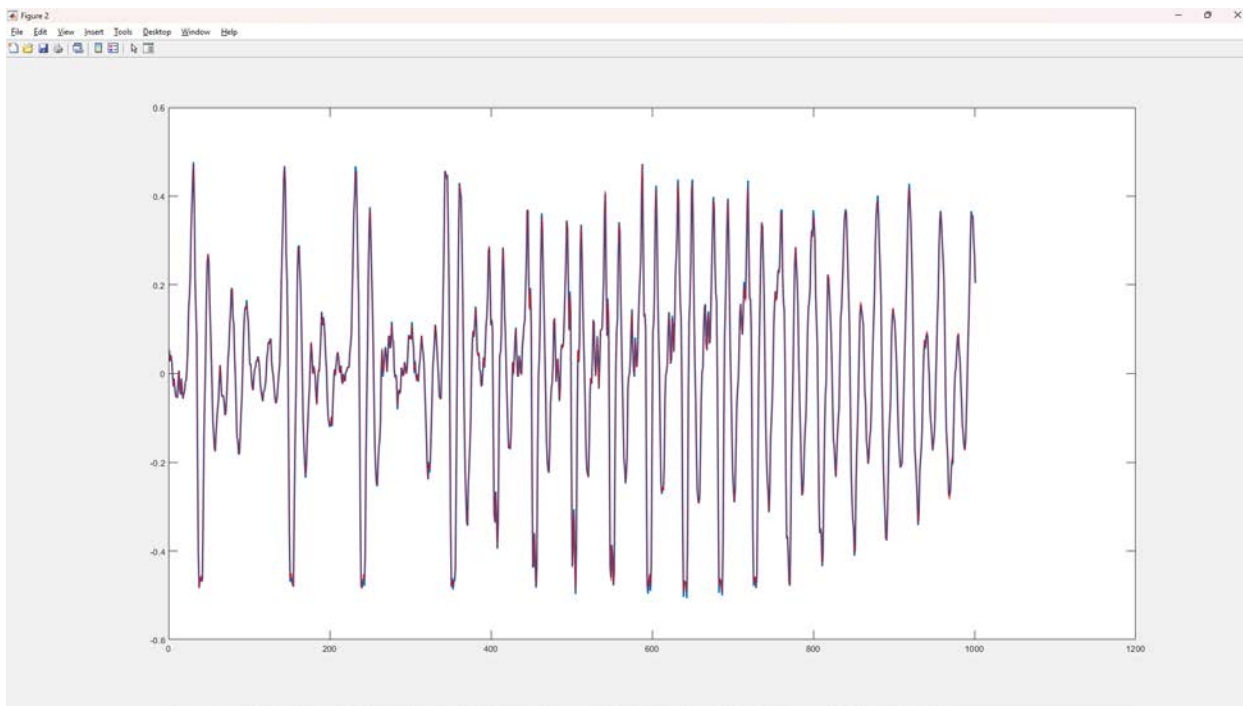
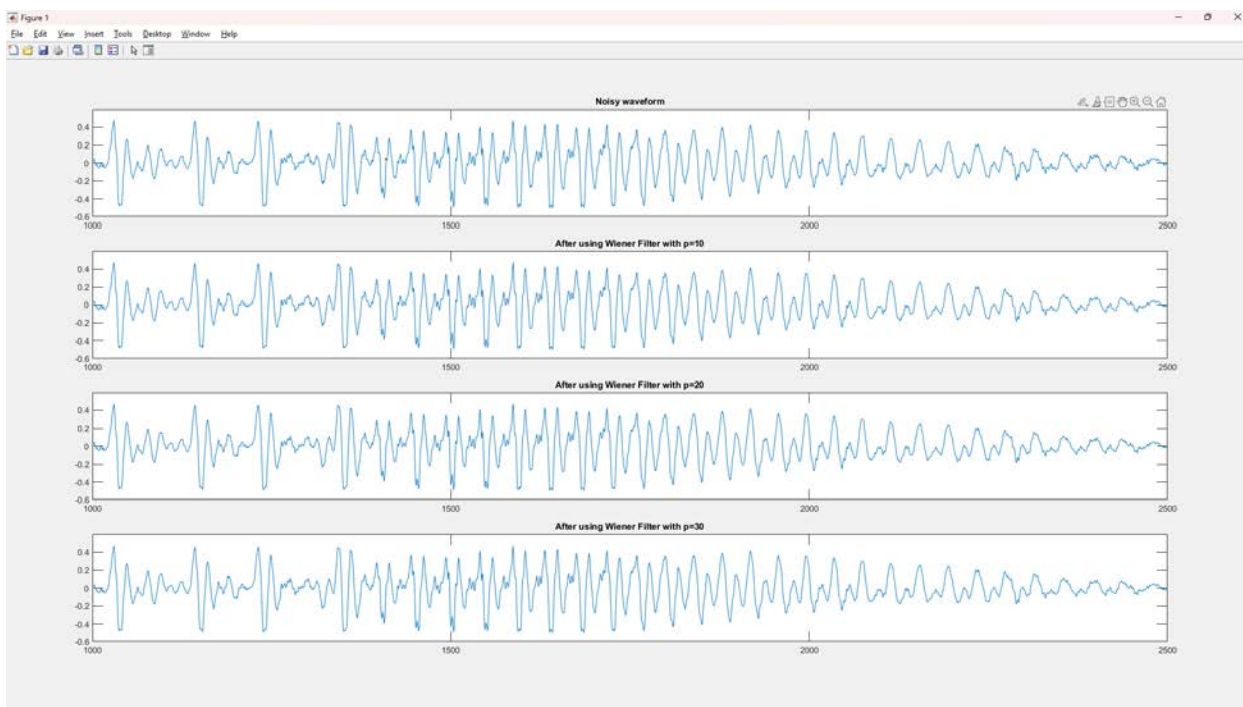
Στην ουσία η διαδικασία είναι όπως η προηγούμενη για κάθε φίλτρο, με την διαφορά να είναι ότι αντί να δημιουργήσουμε ένα φίλτρο Wiener με βάση όλο το σήμα, χωρίζουμε το σήμα σε διανύσματα μήκους 256 δειγμάτων και μέσω μίας δομής επανάληψης για όλα τα διανύσματα δημιουργούμε ένα νέο φίλτρο και εφαρμόζουμε στο υποδιάστημα της κάθε επανάληψης.

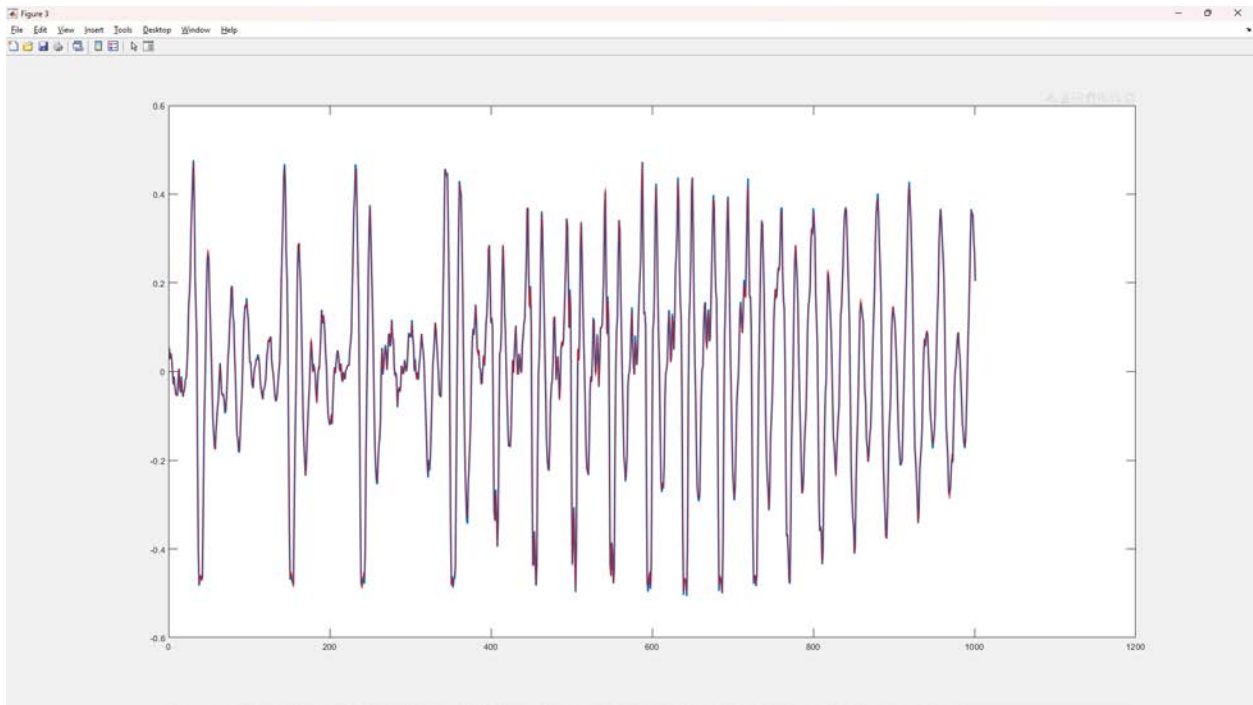
Στο τέλος εξάγονται τα φιλτραρισμένα σήματα σε μορφή αρχείων wav και τυπώνονται τα παρακάτω plots:

```

77 - figure
78 - tiledlayout(4,1);
79 - ax1 = nexttile;
80 - plot(ax1, 750:2750, noisy_y(750:2750))
81 - axis([1000, 2500, -0.6, 0.6])
82 - title(ax1, 'Noisy waveform')
83
84 - ax2 = nexttile;
85 - plot(ax2, 750:2750, denoised_10(750:2750))
86 - axis([1000, 2500, -0.6, 0.6])
87 - title(ax2, 'After using Wiener Filter with p=10')
88
89 - ax3 = nexttile;
90 - plot(ax3, 750:2750, denoised_20(750:2750))
91 - axis([1000, 2500, -0.6, 0.6])
92 - title(ax3, 'After using Wiener Filter with p=20')
93
94 - ax4 = nexttile;
95 - plot(ax4, 750:2750, denoised_30(750:2750))
96 - axis([1000, 2500, -0.6, 0.6])
97 - title(ax4, 'After using Wiener Filter with p=30')
98
99
100 - figure; plot(noisy_y(1000:2000), 'LineWidth', 2), hold on; plot(denoised_10(1000:2000), 'r'), hold off;
101 - figure; plot(noisy_y(1000:2000), 'LineWidth', 2), hold on; plot(denoised_30(1000:2000), 'r'), hold off;

```





Το figure 1 είναι ανάλογο με αυτό του πρώτου ερωτήματος, ενώ στα 2 και 3 συγκρίνονται οι έξοδοι των φίλτρων με τάξεις 10 και 20 με το αρχικό σήμα.

Αν γίνει κατάλληλη μεγέθυνση στο figure 2 που συγκρίνεται το φίλτρο με τάξη 10 με το αρχικό ενθόρυβο σήμα παρατηρούμε ότι τις απότομες κορυφές τις κάνει ελαφρώς πιο ομαλές. Στο figure 3 επειδή το φίλτρο είναι μεγαλύτερης τάξης οι κορυφές γίνονται ακόμα πιο ομαλές οπότε είναι ακόμα πιο φανερό (πάντα μεγεθύνοντας κατάλληλα).

Τα SNR του ενθόρυβου και των φιλτραρισμένων σημάτων από τα φίλτρα τάξης 10, 20 και 30 κατά σειρά είναι τα παρακάτω:

```
>> snr_vec
```

```
snr_vec =
```

```
19.356519059539792 21.406071764725105 21.451095452185491 21.452378791283007
```

Εδώ η βελτίωση είναι φανερή σε σχέση με το αρχικό ενθόρυβο σήμα.

Μάλιστα ακούγοντας τα αρχεία υπάρχει και στον ακροατή φανερή βελτίωση, καθώς σε σημεία που δεν υπάρχει ομιλία, ο θόρυβος εξαλείφεται σχεδόν πλήρως, ενώ και στα σημεία που υπάρχει φωνή υπάρχει μία βελτίωση, αλλά όχι το ίδιο καλή αφού υπάρχει ακόμα θόρυβος στο background.

Ερώτημα C

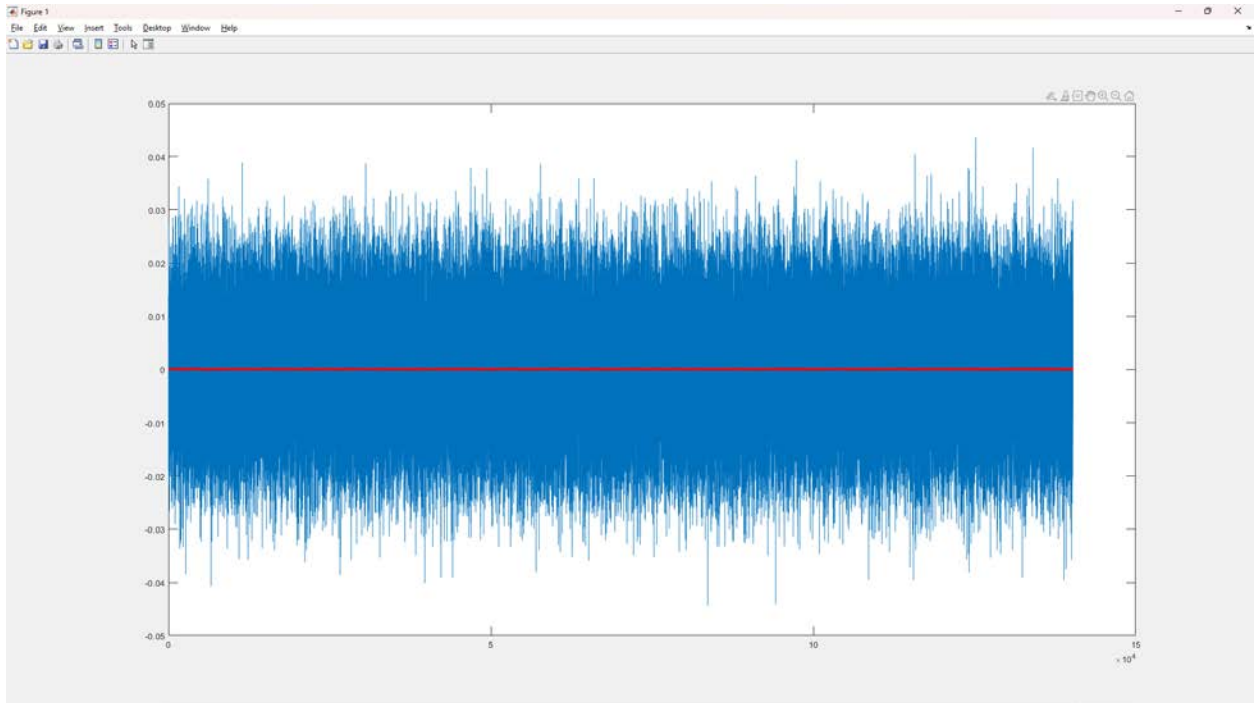
Για το ερώτημα c γράφηκε το παρακάτω script αρχείο που υλοποιεί και τα 1 φίλτρο Wiener με τάξη $p=10$ από το οποίο περνάει τον θόρυβο και στην συνέχεια υπολογίζει και την αρχική μέση τιμή του θορύβου:

```

1 - clear; clc; close all
2 - %Task c
3 - %=====
4 - %Load wav file
5 - [y, Fs] = audioread('guit1.wav');
6 - white_noise = audioread('white_noise.wav');
7 - n = length(y);
8 -
9 - p = 10;
10 -
11 - rss = xcorr(white_noise);
12 - rxx = rss(n:n+p-1);
13 - rss = rss(n+1:n+1+p-1);
14 -
15 - Rxx = toeplitz(rxx);
16 - w = Rxx\rss;
17 - z = filter(w, 1, white_noise);
18 -
19 - fprintf("Mean value of noise is:");
20 - disp(round(mean(white_noise),5));
21 - figure; plot(white_noise); hold on; plot(z, 'r'); hold off;

```

Το διάγραμμα που τυπώνεται και συγκρίνεται το αρχικό σήμα με το φιλτραρισμένο είναι αυτό:



Και η μέση τιμή που τυπώνεται, στρογγυλοποιημένη σε 5 δεκαδικά ψηφία, είναι η παρακάτω:

```
Command Window
Mean value of noise is:    0
```

Συνεπώς ο θόρυβος είναι λευκός.

Ερώτημα D

Η υλοποίηση είναι η παρακάτω:

```

1 - clear; clc; close all
2   %Task d
3   %Load wav files
4 - [y, Fs] = audioread('guit1.wav');
5 - white_noise = audioread('white_noise.wav');
6 - noisy_y = audioread('noisy.wav');
7 - nl = 800;
8 - n2 = length(y);
9 - snr_vec = 20*log10(norm(y)/norm(noisy_y-y));
10
11 - noisy_est = noisy_y(1:800);
12
13   %Wiener order p=10
14 - p=10;
15 - rnoise = xcorr(noisy_est);
16 - rxx = xcorr(noisy_y);
17 - rnoise = rnoise(nl:nl+p-1); %rnn
18 - rxx = rxx(n2:n2+p-1); %rxx
19 - rxd = rxx-rnoise; %rxd
20
21 - Rxx = toeplitz(rxx);
22 - w = Rxx\rxd;
23
24 - z10 = filter(w, 1, noisy_y);
25 - snr_vec = [snr_vec 20*log10(norm(y)/norm(z10-y))];
26 - audiowrite('wiener10_d.wav', z10, Fs);
27
28
29   %Wiener order p=20
30 - p = 20;
31 - rnoise = xcorr(noisy_est);
32 - rxx = xcorr(noisy_y);
33 - rnoise = rnoise(nl:nl+p-1); %rnn
34 - rxx = rxx(n2:n2+p-1); %rxx
35 - rxd = rxx-rnoise; %rxd

```

```

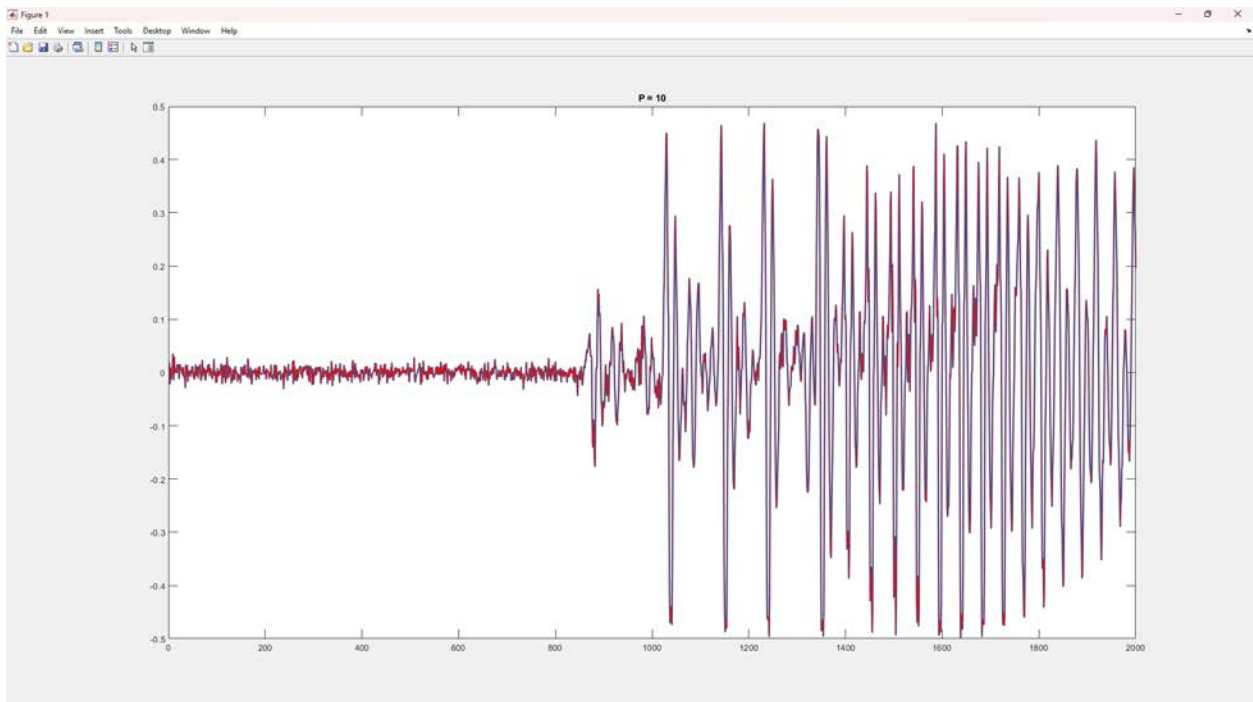
36
37 - Rxx = toeplitz(rxx);
38 - w = Rxx\rxd;
39
40
41 - z20 = filter(w, 1, noisy_y);
42 - snr_vec = [snr_vec 20*log10(norm(y)/norm(z20-y))];
43 - audiowrite('wiener10_d.wav', z20, Fs);
44
45 %Wiener order p=30
46 - p = 30;
47 - rnoise = xcorr(noisy_est);
48 - rxx = xcorr(noisy_y);
49 - rnoise = rnoise(n1:n1+p-1); %rnn
50 - rxx = rxx(n2:n2+p-1); %rxx
51 - rxd = rxx-rnoise; %rxd
52
53 - Rxx = toeplitz(rxx);
54 - w = Rxx\rxd;
55
56 - z30 = filter(w, 1, noisy_y);
57 - snr_vec = [snr_vec 20*log10(norm(y)/norm(z30-y))];
58 - audiowrite('wiener10_d.wav', z30, Fs);
59
60 - figure; plot(noisy_y(1000:2000), 'LineWidth', 2), hold on; plot(z10(1000:2000), 'r'), hold off; title('P = 10');
61 - figure; plot(noisy_y(1000:2000), 'LineWidth', 2), hold on; plot(z20(1000:2000), 'r'), hold off; title('P = 20');
62 - figure; plot(noisy_y(1000:2000), 'LineWidth', 2), hold on; plot(z30(1000:2000), 'r'), hold off; title('P = 30');

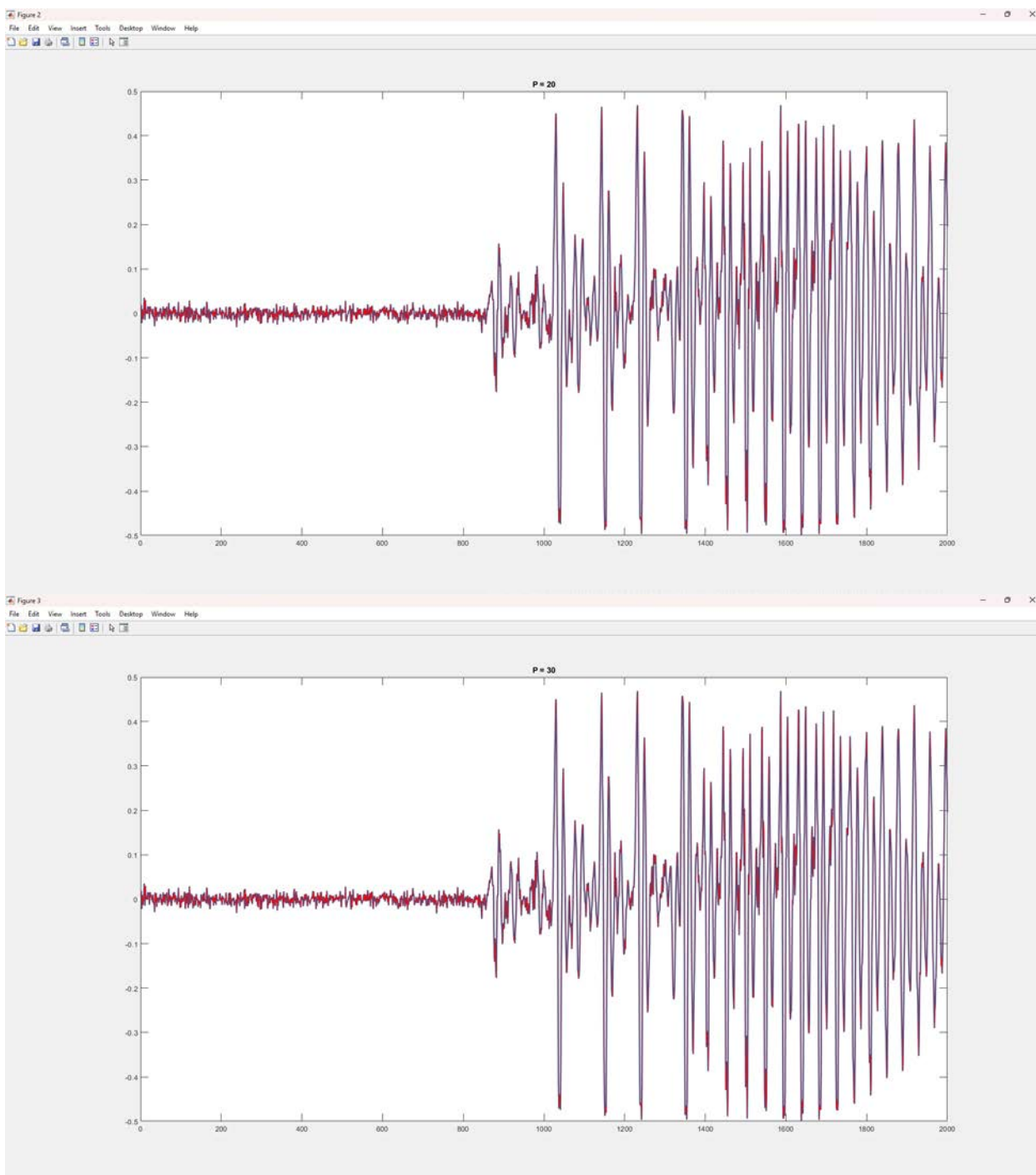
```

Η διαφοροποίηση στην υλοποίηση εδώ είναι ότι δημιουργώ την **noisy_est**. Η **noisy_est** περιέχει το ενθόρυβο σήμα μέχρι το 800οστο δείγμα. Βρίσκω τις αυτοσυσχετίσεις του **noisy_est** και όλου του ενθόρυβου κρατάω του p συντελεστές που με ενδιαφέρουν και τις αφαιρώ μεταξύ τους.

Και με αυτό τον τρόπο υλοποιώ την μεταβλητή **rxd** που την χρησιμοποιώ για να δημιουργήσω το φίλτρο που θα εφαρμόσω στο σήμα.

Παρακάτω βλέπω το αποτέλεσμα για κάθε τιμή του p σε ένα μέρος του σήματος:





Το SNR του σήματος δίνεται (με την ίδια σειρά) παρακάτω:

```
>> snr_vec  
  
snr_vec =  
  
19.381931258618359 19.385854304368721 19.385948389450959 19.385993250953447
```

Παρατηρώ ότι όσο μεγαλύτερο είναι το p τόσο καλύτερα τα αποτελέσματα, αλλά και πάλι, όπως συνέβη και στο α) ερώτημα η διαφορά δεν είναι ιδιαίτερα εμφανής.

Ακούγοντας τα φιλτραρισμένα αρχεία ήχου, αυτό παρατηρείται είναι ότι παρά την βελτίωση που υπάρχει σύμφωνα με τις τιμές του SNR η διαφορά δεν είναι αρκετή για να γίνει αντιληπτή από το ανθρώπινο αφτί.

Μάλιστα σε σύγκριση με τα αποτελέσματα του ερωτήματος α) τα SNR σε αυτό το ερώτημα έχουν χαμηλότερη τιμή, το οποίο είναι λογικό αφού εδώ δεν είχαμε γνώση του επιθυμητού σήματος και αποθορυβοποιήσαμε το σήμα με χρήση κομματιού του ίδιου του ενθόρυβου σήματος.

Ερώτημα Ε

Η υλοποίηση:

```

1 - clear; clc; close all;
2 - %Task e
3 - %-----
4 - %Load wav file
5 - [y, Fs] = audioread('guilt1.wav');
6 - noisy_y = audioread('noisy.wav');
7 - n = length(y);
8 -
9 - snr_vec = 20*log10(norm(y)/norm(noisy_y-y));
10 - win_size = 256;
11 -
12 - frames_noisy = frame_wind(noisy_y, win_size, 0.5);
13 -
14 - % Wiener of order p = 10
15 - p = 10;
16 - rxx = xcorr(noisy_y);
17 - rxx = rxx(n:n+p-1);
18 - Rxx = toeplitz(rxx);
19 -
20 - for i=1:size(frames_noisy, 2)
21 -     if mean(frames_noisy(:,i).^2) < 0.0001
22 -         rnoise = xcorr(frames_noisy(:,i));
23 -         rnoise = rnoise(win_size:win_size+p-1);
24 -
25 -         rxd_noise = rnoise-rnoise;
26 -         rxd = rxx-rnoise;
27 -
28 -         w_noise = pinv(Rxx,0.0001)*rxd_noise;
29 -         w = pinv(Rxx,0.0001)*rxd;
30 -
31 -         z(:, i) = filter(w_noise, 1, frames_noisy(:,i));
32 -     else
33 -         z(:, i) = filter(w, 1, frames_noisy(:,i));
34 -     end
35 - end
36 -
37 - denoised_10 = frame_recon(z, 0.5)';
38 - denoised_10 = denoised_10(1:140278);
39 - snr_vec = [snr_vec 20*log10(norm(y)/norm(denoised_10-y))];
40 - figure; plot(noisy_y(1:2000), 'LineWidth', 2), hold on; plot(denoised_10(1:2000), 'r'), hold off; title("P = 10");
41 - audiowrite('wiener10_e.wav', denoised_10, Fs);
42 -
43 - %-----
44 - % Wiener of order p = 20
45 - p = 20;
46 - rxx = xcorr(noisy_y);
47 - rxx = rxx(n:n+p-1);
48 - Rxx = toeplitz(rxx);
49 -
50 - for i=1:size(frames_noisy, 2)
51 -     if mean(frames_noisy(:,i).^2) < 0.0001
52 -         rnoise = xcorr(frames_noisy(:,i));
53 -         rnoise = rnoise(win_size:win_size+p-1);
54 -
55 -         rxd_noise = rnoise-rnoise;
56 -         rxd = rxx-rnoise;
57 -
58 -         w_noise = pinv(Rxx,0.0001)*rxd_noise;
59 -         w = pinv(Rxx,0.0001)*rxd;
60 -
61 -         z(:, i) = filter(w_noise, 1, frames_noisy(:,i));
62 -     else
63 -         z(:, i) = filter(w, 1, frames_noisy(:,i));
64 -     end
65 - end
66 -
67 - denoised_20 = frame_recon(z, 0.5)';
68 - denoised_20 = denoised_20(1:140278);
69 - snr_vec = [snr_vec 20*log10(norm(y)/norm(denoised_20-y))];
70 - figure; plot(noisy_y(1:2000), 'LineWidth', 2), hold on; plot(denoised_20(1:2000), 'r'), hold off; title("P = 20");
71 - audiowrite('wiener20_e.wav', denoised_20, Fs);
72 -

```

```

73 %-----
74 % Wiener of order p = 30
75 p = 30;
76 rxx = xcorr(noisy_y);
77 rxx = rxx(n:n+p-1);
78 Rxx = toeplitz(rxx);
79
80 for i=1:size(frames_noisy, 2)
81     if mean(frames_noisy(:,i).^2) < 0.0001
82         rnoise = xcorr(frames_noisy(:,i));
83         rnoise = rnoise(win_size:win_size+p-1);
84
85         rxd_noise = rnoise-rnoise;
86         rxd = rxx-rnoise;
87
88         w_noise = pinv(Rxx,0.0001)*rxd_noise;
89         w = pinv(Rxx,0.0001)*rxd;
90
91         z(:, i) = filter(w_noise, 1, frames_noisy(:,i));
92     else
93         z(:, i) = filter(w, 1, frames_noisy(:,i));
94     end
95 end
96
97 denoised_30 = frame_recon(z, 0.5)';
98 denoised_30 = denoised_30(1:140278);
99 snr_vec = [snr_vec 20*log10(norm(y)/norm(denoised_30-y))];
100 figure; plot(noisy_y(1:2000), 'r', hold on; plot(denoised_30(1:2000), 'r', hold off; title("P = 30");
101 audiowrite('wiener30 e.wav', denoised_30, Fs);

```

Αρχικά ορίζω τον πίνακα Rxx καθώς αυτός θα βασίζεται σε ολόκληρο το ενθόρυβο σήμα οπότε δεν χρειάζεται να το υπολογίζω σε κάθε επανάληψη.

Στην συνέχεια ξεκινάει η δομή επανάληψης.

Ο τρόπος με τον οποίο αναγνωρίζω αν ένα παράθυρο έχει φωνή ή μόνο θόρυβο είναι ο εξής:

1. Υπολογίζω το τετράγωνο του κάθε στοιχείου του παραθύρου.
2. Βρίσκω τον μέσο όρο των νέων στοιχείων.
3. Συγκρίνω με το 0.0001 και αν είναι μικρότερο θεωρώ ότι το παράθυρο έχει μόνο θόρυβο.

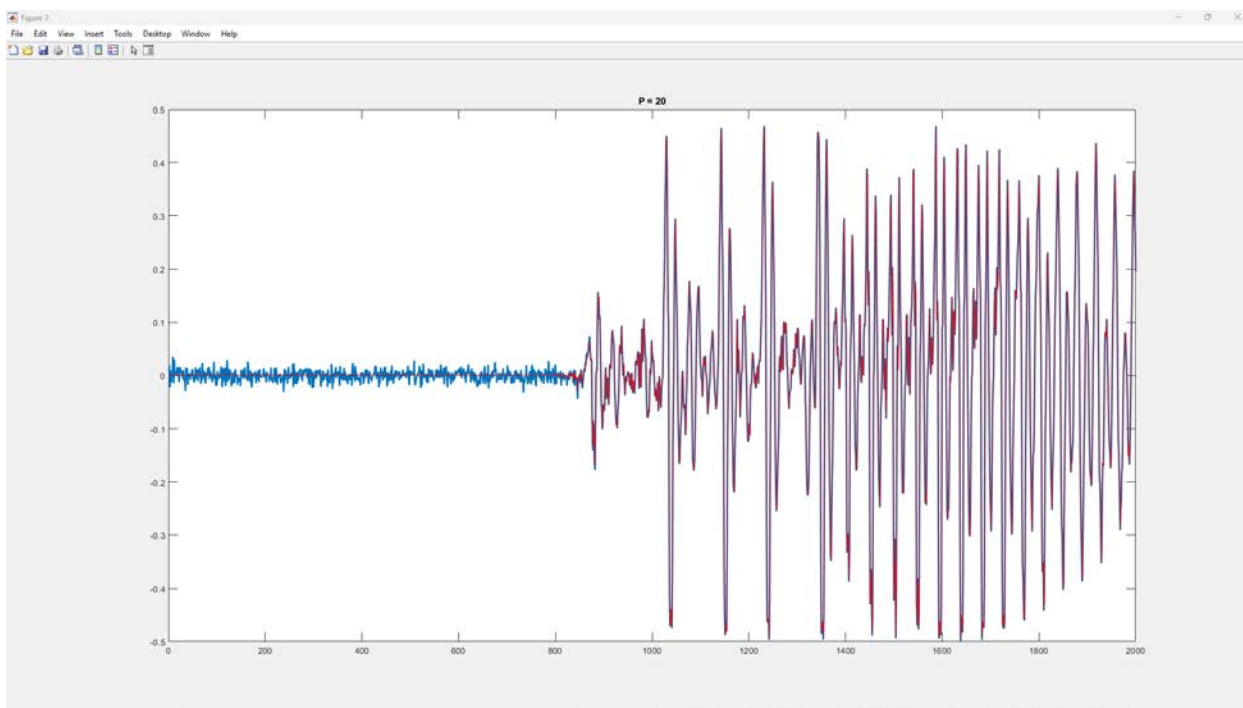
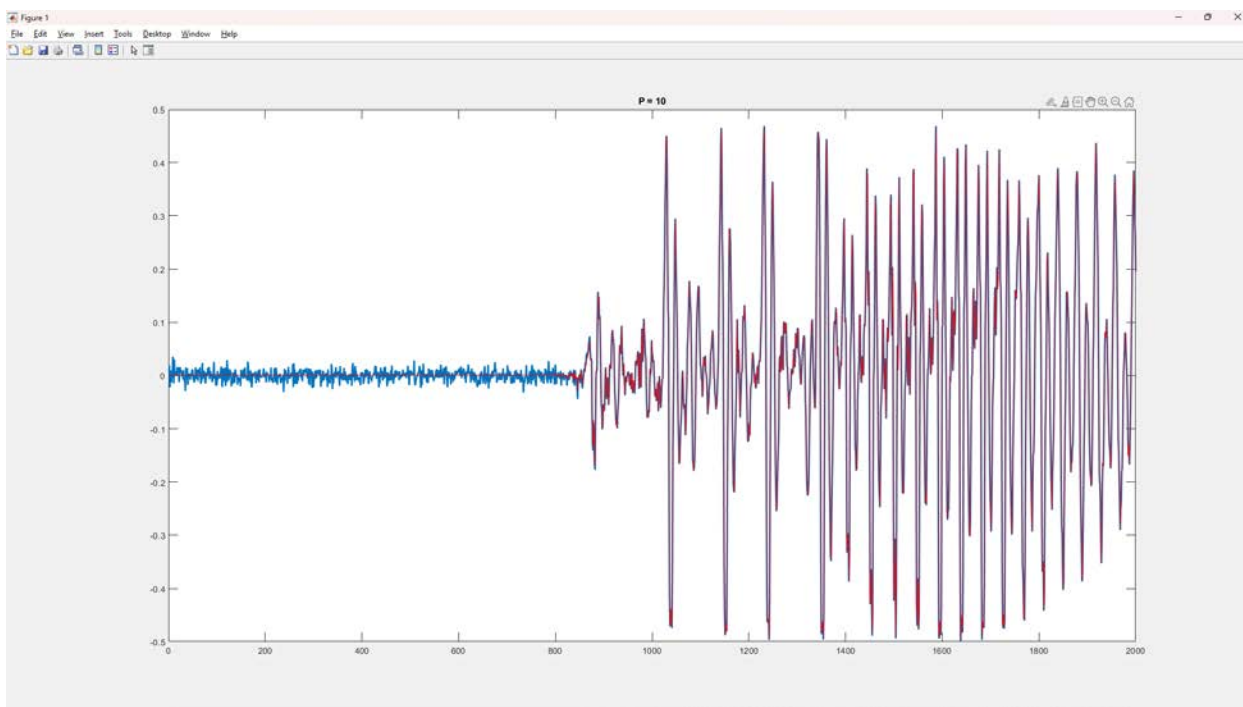
Κάθε φορά που βρίσκω ένα αθόρυβο παράθυρο, ανανεώνω την τιμή rnoise (αντίστοιχο του rnh των διαφανειών της θεωρίας)

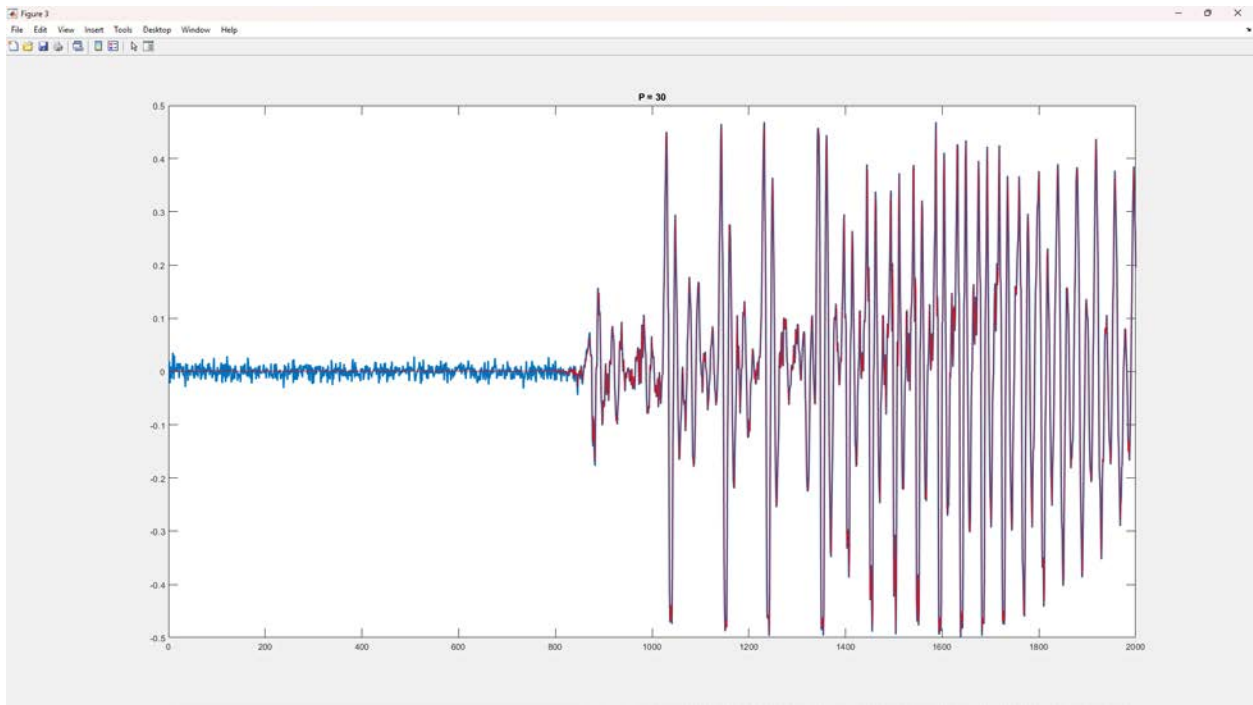
Στα αθόρυβα παράθυρα χρησιμοποιώ ως rxd το rxd_noise το οποίο είναι ένα μηδενικό διάνυσμα και με αυτό το τρόπο τα παράθυρα με θόρυβο μηδενίζονται τελείως.

Ενώ για τα υπόλοιπα παράθυρα σε κάθε ανανέωση του αθόρυβου παραθύρου υπολογίζεται το νέο rxd που θα χρησιμοποιείται για την δημιουργία φίλτρου.

Όταν το παράθυρο δεν είναι αθόρυβο τότε χρησιμοποιείται το απλό φίλτρο w που υλοποιείται με χρήση της μεταβλητής rxd (όχι της rxd_noise).

Τυπώνω και ένα μέρος των νέων διαγραμμάτων σε σύγκριση με το ενθόρυβο για κάθε φίλτρο ξεχωριστά:





Το SNR των φιλτραρισμένων σημάτων είναι το παρακάτω (θέσεις 2, 3 και 4, για $p=10$, $p=20$ και $p=30$ αντίστοιχα):

```
>> snr_vec

snr_vec =

    19.381931258618359    19.998002385633981    19.998013847695848    19.998025679401010
```

Όπως συνέβη και στα ερωτήματα α) και β), έτσι και στα ερωτήματα δ) και ε) παρατηρούμε ότι τα αποτελέσματα του ερωτήματος ε είναι καλύτερα. Βέβαια και πάλι δεν ήταν δυνατή η επίτευξη το ίδιο καλής βελτίωσης.

Μάλιστα λόγω της χρήσης του `rx_d_noise` κατά την ακρόαση των ηχητικών αρχείων, παρόλο που στα κομμάτια που δεν υπάρχει ομιλία εντοπίζουμε πλήρη αποθορυβοποίηση, το αποτέλεσμα είναι αρκετά αφύσικο και όχι ιδιαίτερα ευχάριστο στο ανθρώπινο αφτί.

Ερώτημα F

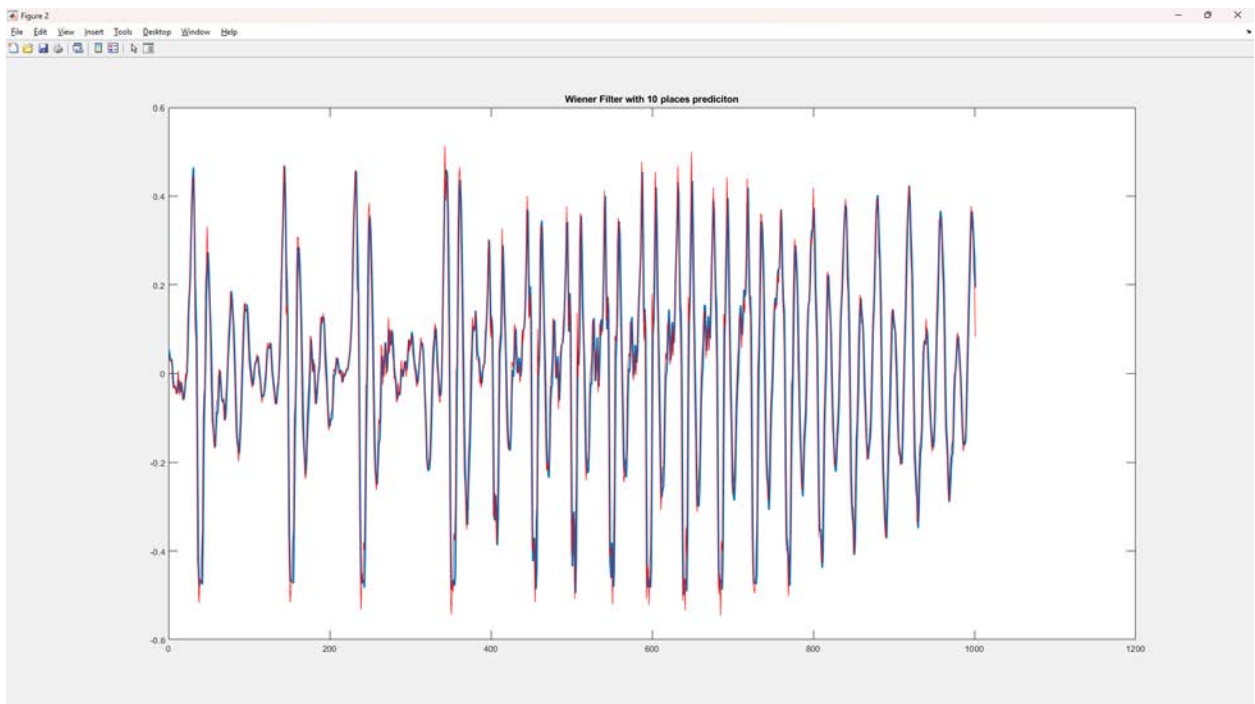
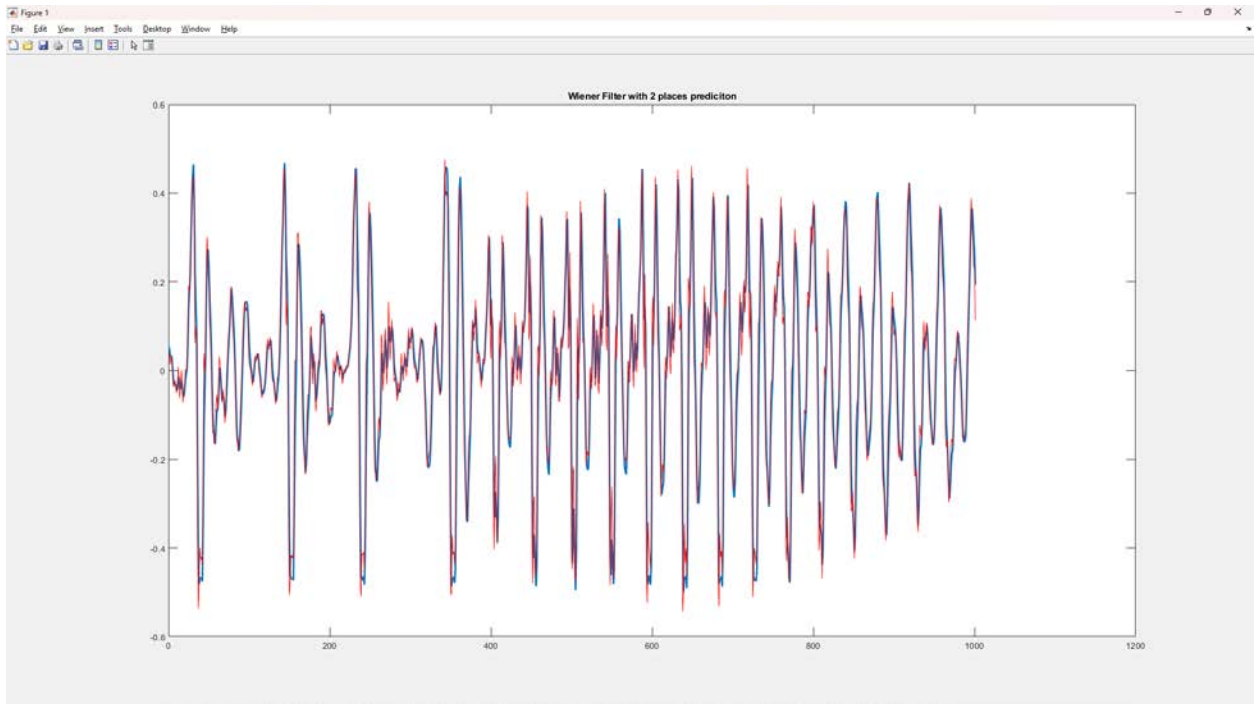
```

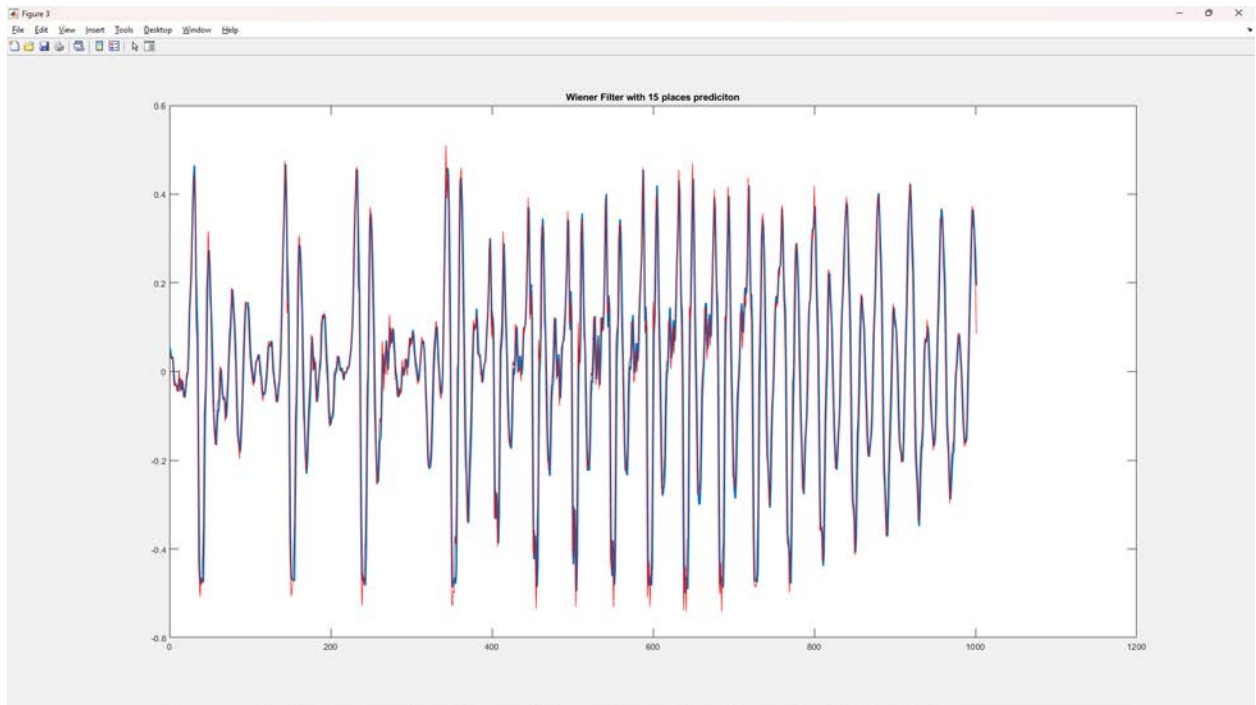
1 - clear; clc; close all
2 - %Task f
3 - %=====
4 - %Load wav file
5 - [y, Fs] = audioread('guit1.wav');
6 - n = length(y);
7 - win_size = 256;
8 - frames_original = frame_wind(y, win_size, 0.5);
9 -
10 - %Wiener 2 positions
11 - for i=1:size(frames_original, 2)
12 -
13 -     rss = xcorr(frames_original(:,i));
14 -     rxx = rss(256:256+2);
15 -     rss = rss(257:257+2);
16 -
17 -     Rxx = toeplitz(rxx);
18 -
19 -     w = pinv(Rxx, 0.0001)*rss; % It is the same as inv(Rxx)*rxd
20 -     z(:, i) = filter(w, 1, frames_original(:,i));
21 - end
22 -
23 - pred_2 = frame_recon(z, 0.5)';
24 - figure; plot(y(1000:2000), 'LineWidth', 2); hold on; plot(pred_2(1000:2000), 'r'), hold off;
25 - title("Wiener Filter with 2 places predicition");
26 -
27 - %Wiener 10 positions
28 - for i=1:size(frames_original, 2)
29 -
30 -     rss = xcorr(frames_original(:,i));
31 -     rxx = rss(256:256+10);
32 -     rss = rss(257:257+10);
33 -
34 -     Rxx = toeplitz(rxx);
35 -
36 -     w = pinv(Rxx, 0.0001)*rss; % It is the same as inv(Rxx)*rxd
37 -     z(:, i) = filter(w, 1, frames_original(:,i));
38 - end
39 -
40 - pred_10 = frame_recon(z, 0.5)';
41 - figure; plot(y(1000:2000), 'LineWidth', 2), hold on; plot(pred_10(1000:2000), 'r'), hold off;
42 - title("Wiener Filter with 10 places predicition");
43 -
44 - %Wiener 15 positions
45 - for i=1:size(frames_original, 2)
46 -
47 -     rss = xcorr(frames_original(:,i));
48 -     rxx = rss(256:256+15);
49 -     rss = rss(257:257+15);
50 -
51 -     Rxx = toeplitz(rxx);
52 -
53 -     w = pinv(Rxx, 0.0001)*rss; % It is the same as inv(Rxx)*rxd
54 -     z(:, i) = filter(w, 1, frames_original(:,i));
55 - end
56 -
57 - pred_15 = frame_recon(z, 0.5)';
58 - figure; plot(y(1000:2000), 'LineWidth', 2), hold on; plot(pred_15(1000:2000), 'r'), hold off;
59 - title("Wiener Filter with 15 places predicition");
60 -
61 - snr_vec = [20*log10(norm(y(2:n))/norm(pred_2(1:n-1)-y(2:n))), 20*log10(norm(y(2:n))/norm(pred_10(1:n-1)-y(2:n))),
62 -            20*log10(norm(y(2:n))/norm(pred_15(1:n-1)-y(2:n)))];

```

Εδώ χρησιμοποιήθηκε μέσα στην δομή επανάληψης ο κώδικας που δίνεται και στις διαφάνειες της διάλεξης.

Μετά από κάθε διαφορετικό φίλτρο Wiener που υλοποιείται για την πρόβλεψη του σήματος εισόδου τυπώνουμε και πάλι μέρος των κυματομορφών του αρχικού σήματος και της κάθε εξόδου του φίλτρου για να γίνει σύγκριση (λαμβάνοντας υπόψη την μετατόπιση):





Στο τέλος του αρχείου υπολογίζονται και τα SNR για την κάθε έξοδο:

```
>> snr_vec

snr_vec =

    14.398497275166513    16.841107736124691    17.600819892221281
```

Και πάλι όσο μεγαλώνει η τιμή p (δεν δίνεται σαν μεταβλητή, αλλά το εννοείται ανάλογα και με τα προηγούμενα ερωτήματα) τόσο καλύτερες οι τιμές του SNR για το σήμα που δίνεται ως έξοδος του κάθε φίλτρου.

Ακούγοντας τα αρχεία εξόδου, παρατηρήθηκε ότι το σήμα που πρόβλεψε το φίλτρο είχε μία φανερή αλλοίωση (βέβαια όσο μεγαλύτερη η τιμή του p τόσο λιγότερο αισθητή). Παρόλα αυτά η φωνή ακουγόταν, με την έννοια ότι τα λόγια που εκφωνήθηκαν μπορούσαν να γίνουν κατανοητά με ευκολία)

Σημείωση

Πριν τρέξετε οποιοδήποτε από τα αρχεία της εργασίας, εκτελέστε μία φορά το **Noisy_wave_creation.m** (δεν χρειάζεται να το ξαναεκτελέσετε αν θέλετε να τρέξετε πάνω από ένα αρχείο).